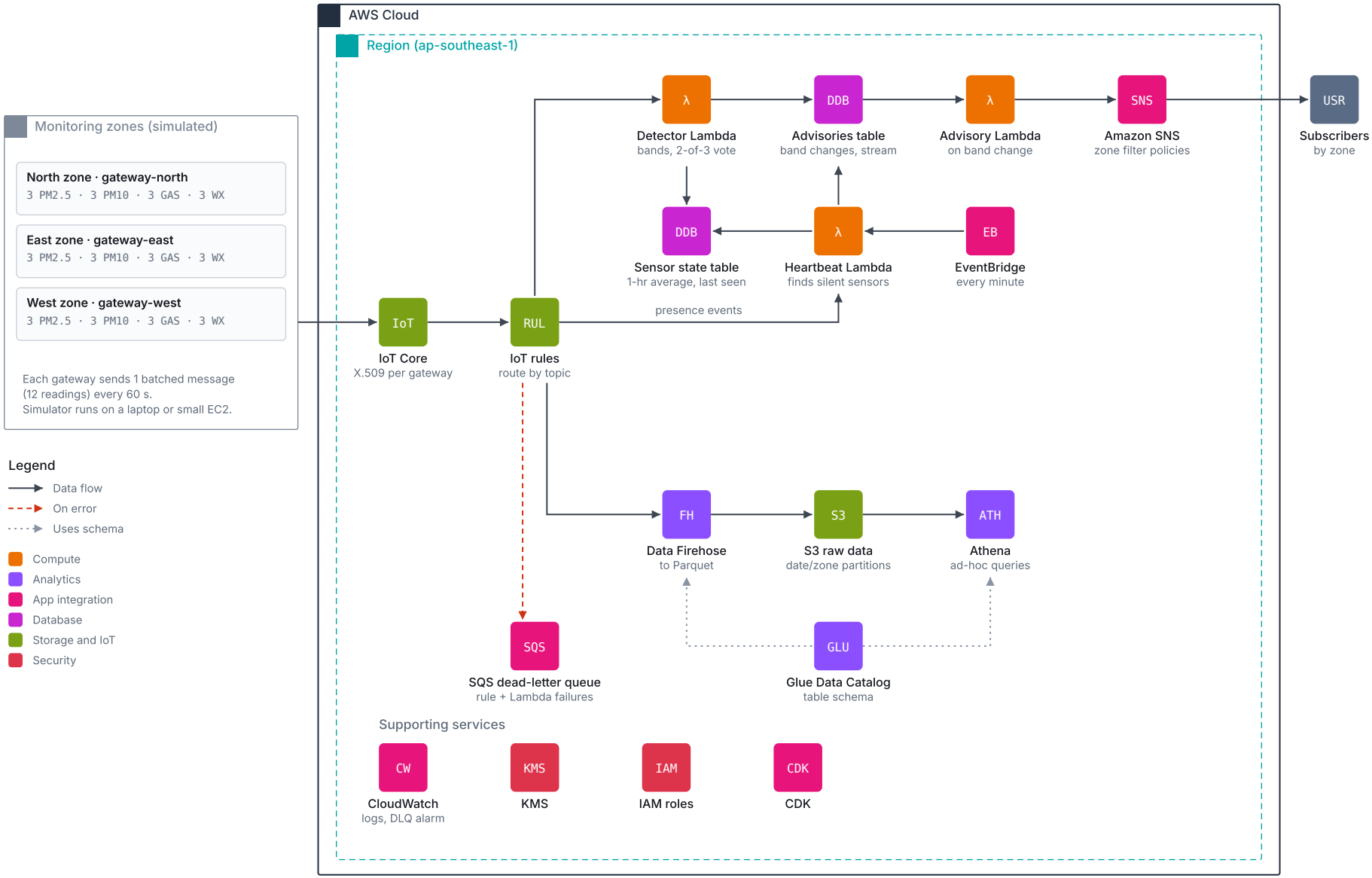
Real-time haze monitoring on AWS · Phase 1 architecture

3 zones (North, East, West) × (3 PM2.5, 3 PM10, 3 gas, 3 weather) = 36 simulated sensors · serverless, no VPC · ap-southeast-1



All sensor values are synthetic. Bands follow NEA's 1-hr PM2.5 bands.

PORTFOLIO PROJECT · IOT AND STREAMING · PHASE 1 OF 3

Real-time haze monitoring on AWS

Singapore's haze season can push air quality from normal to unhealthy within hours, and conditions differ across the island. This system collects readings from air-quality sensors in three zones and sends residents a zone-specific advisory when PM2.5 crosses NEA's bands, without being fooled by a single faulty sensor.

Requirements

Functional

- Ingest readings from 36 sensors in 3 zones every 60 seconds.
- Track a 1-hour PM2.5 average per sensor and map it to NEA's 4 bands.
- Send one advisory per zone when its band changes; subscribers choose zones.
- Ignore a single faulty sensor; detect sensors and gateways that stop reporting.
- Keep full history for later analysis.

Non-functional

- Advisory within 2 minutes of a band change.
- Survive the loss of an Availability Zone.
- No lost messages: failures are captured, not dropped.
- Encrypted in transit and at rest; least-privilege IAM.
- Under $15 per month running 24/7.

NEA 1-hr PM2.5 bands (µg/m³)

Band	Descriptor	Range
I	Normal	0–55
II	Elevated	56–150
III	High	151–250
IV	Very high	251 and above

How it works

1. Ingest

1. Each zone's 12 sensors report to that zone's gateway.
2. Every 60 s each gateway batches its readings into one MQTT message, over TLS with its own X.509 certificate.
3. IoT Core receives the batch; IoT rules route it by topic.

2. Zone advisories

1. The detector updates each sensor's 1-hour average and maps it to a band. Writes are conditional on sequence number, so duplicates and late messages are ignored.
2. The zone's band changes only when 2 of its 3 PM2.5 sensors agree.
3. Band changes go to the advisories table, whose stream triggers the advisory Lambda.
4. SNS sends the advisory with a zone attribute; subscriber filter policies pick zones.

3. Offline detection

1. EventBridge runs the heartbeat Lambda every minute.
2. It flags any sensor not seen for 3 intervals.
3. IoT Core disconnect events also trigger it, catching a whole zone gateway going down.
4. Offline records reach operators only, through a message-type filter.

4. Data lake and failures

1. Firehose converts each batch to Parquet using the Glue Data Catalog schema, partitioned by date and zone.
2. Athena queries it, unnesting the readings array.
3. Failed rule actions and failed Lambda runs land in an SQS dead-letter queue, with a CloudWatch alarm on its depth.

Design decisions

Decision	Alternative	Why
One gateway per zone, batched messages	One connection and message per sensor	3 certificates instead of 36, and 12 times fewer messages, rule runs and Lambda invocations. Mirrors how field sensors reach the network through a gateway.
60-second readings	10-second readings	Haze builds over hours and NEA bands use 1-hour averages. 60 s is ample and cuts cost about 6 times.
2-of-3 PM2.5 sensor vote per zone	Any sensor over the limit	One faulty or dirty sensor must never trigger a public advisory. Batching per zone puts all 3 votes in one message.
Exponentially weighted 1-hour average	Store 60 readings per sensor	One number per sensor gives a close approximation of the 1-hour average, with one write per reading.
Separate advisories table with a stream	One table with a stream	A stream fires on every write. Writing only band changes means one advisory per change, not per reading.
SNS filter policies by zone and message type	One topic per zone	Subscribers choose zones on one topic; operators alone receive sensor-offline messages.
IoT rule invokes Lambda directly	Kinesis Data Streams	3 messages per minute needs no buffering, and Kinesis adds a fixed hourly cost. Switch at thousands per second or for replay.
Conditional writes on sequence number	Last write wins	MQTT QoS 1 and Lambda retries can deliver duplicates or out of order.
Firehose to Parquet with dynamic partitioning	Raw JSON in S3	Athena scans far less data; partitions by date and zone keep queries cheap.
DLQ for rule errors and Lambda failures	Rule error action only	The rule's error action catches failed invocations, not errors inside the function.
Serverless, no VPC	VPC with private subnets	Every service is regional and multi-AZ by default. A VPC adds NAT cost with no security gain here.

Known gaps: scale and cost trade-offs

Gap	Why it is fine now	What changes at scale
No Kinesis	Tiny message rate, no fixed cost	Add Kinesis for high throughput, ordering or replay
Heartbeat scans the table	36 items is a trivial scan	GSI on last-seen, or shard by gateway
Sensor calibration and humidity correction not modelled	Synthetic data	Per-sensor calibration factors from co-located reference stations
No gateway buffering during outages	It is a simulator	Store-and-forward on the gateway, e.g. AWS IoT Greengrass
Single region	All services are multi-AZ	Multi-region DR (phase 3)
No public dashboard or self-service subscriptions	Phase 1 proves the core	Phase 2

Cost estimate

Rough monthly estimate running 24/7, from list prices. Singapore rates are slightly higher than US rates; confirm in the AWS Pricing Calculator.

Service	Driver	Approx. per month
IoT Core	~130k batched messages plus rule triggers and actions	$0.30
Lambda	~130k detector runs, heartbeat every minute	$0.10
DynamoDB on-demand	~1.6M sensor-state writes	$2–3
Firehose, S3, Athena	Small volumes	$0.50
CloudWatch Logs	Errors and advisories only, 7-day retention	$0.50
SNS email, SQS, EventBridge	Very low volume	~$0
Total		**$3–5**

Cost guardrails

- [ ] AWS Budgets alerts at $10 and $15 (they notify, they do not stop spend).
- [ ] Cost Anomaly Detection turned on.
- [ ] Simulator runs on a laptop, not EC2.
- [ ] AWS-managed KMS keys, or one customer-managed key ($1 each).
- [ ] Project=haze-monitoring cost allocation tag activated.

Roadmap

Phase	Build	Proves
1. Working core	Simulator, IoT Core, detector and advisory Lambdas, DynamoDB, SNS with filter policies, offline detection, Firehose, S3, Glue, Athena	Event-driven design and data lakes on a small budget
2. Public features	Amplify dashboard with each zone's band, self-service zone subscriptions, Cognito for operators, Step Functions advisory workflow (escalate, all-clear)	Workflow design, auth, APIs
3. Scale-up	VPC, RDS station registry, OpenSearch, EMR Serverless, Multi-AZ failover tests, comparison with NEA's public readings. Run only during tests.	Networking, high availability, real data

Build checklist: phase 1

- [ ] CDK app in Python; stacks for ingest, advisories, data lake.
- [ ] IoT things, 3 gateway certificates, policies scoped to each zone's topic.
- [ ] IoT rules: detector, Firehose, presence events; error action to the DLQ.
- [ ] Detector, advisory and heartbeat Lambdas with on-failure destinations.
- [ ] Sensor state and advisories tables; stream on advisories only.
- [ ] Firehose to Parquet with the Glue table; Athena workgroup.
- [ ] CloudWatch alarms: DLQ depth, Lambda errors; SNS subscriptions with filters.
- [ ] Tests: single faulty sensor (no advisory), zone haze event (advisory), stuck, offline, gateway disconnect.
- [ ] README: diagram, decisions, cost, test results, teardown.